Mixed fast matrix multiplication: fused levels of accurate square and rectangular schemes over a vendor GEMM, chosen by a calibrated planner

Greg Sidebottom

Draft — October 28, 2026

Abstract

A fast matrix multiplication in practice is a few levels of a Strassen-like scheme over a vendor GEMM. We treat the levels as a menu: every level may use a different scheme, square or rectangular, and a plan is a sequence of levels ending in a GEMM leaf. On Apple's M4 Pro, with Accelerate's GEMM at the leaves, the menu is Strassen–Winograd $\langle 2 \times 2 \times 2 : 7 \rangle$, the 262-operation accurate $\langle 4 \times 4 \times 4 : 48 \rangle$ algorithm of Dumas, Pernet and Sedoglavic, and their accurate $\langle 3 \times 3 \times 6 : 40 \rangle$ algorithm in its three orientations, which bring a square problem back to square after three levels. A level is three streaming passes whose cost is the number of blocks they move, so a scheme's worth is its saving of products per block moved and its amplification of the leaf error; a screen of 70 published schemes by these two measures leaves exactly this menu, and shows that nothing larger than 4×4 and no Kronecker composite can help. A calibrated dynamic program chooses the plan per shape, precision and error budget; it prices GEMM streams and pipelined passes that use the whole machine, and prices rectangular problems from the square calibration until they are calibrated themselves. At $n = 13824$ the planner's mixed plans run 1.73× (f32) and 1.91× (f64) faster than Accelerate's GEMM, with 40% less f32 error than the best plan from the 4×4 scheme alone and less error than the GEMM's own; at $n = 6912$ they run 1.38× and 1.71×. As a test of the opposite design – cheap passes instead of accurate ones – a ternary $\langle 3 \times 3 \times 6 : 42 \rangle$ scheme from Perminov's catalogue, whose three programs need 139 additions against about 370 operations for the accurate scheme's, is measured as a level in the same library: its passes cost exactly the same (they are bound by the blocks they move), and its two extra products and 2.8× amplification make it slower and less accurate than the accurate level at every size tried. Two implementations of the library, in C and in Rust, agree bit for bit.

1 Introduction

A scheme for multiplying a $g_1 \times g_2$ matrix by a $g_2 \times g_3$ matrix in $r < g_1 g_2 g_3$ products, applied to blocks, saves products at every level of recursion. On a real machine a level is not free: its linear combinations of blocks are passes over memory, and the products it hands down are GEMMs whose rate depends on their size and shape. Libraries that use fast schemes therefore apply one to three levels on top of a tuned GEMM and stop [7, 5]. Which scheme, how many levels, in what order, and at what cost in accuracy, are the questions this report answers for one machine, Apple's M4 Pro with Accelerate's GEMM, by measurement.

The approach is to mix. A *plan* is a sequence of *levels* from the top of the problem down to a GEMM leaf, and every level may come from a different scheme: Strassen–Winograd's $\langle 2 \times 2 \times 2 : 7 \rangle$ (`SW`), the accurate $\langle 4 \times 4 \times 4 : 48 \rangle$ algorithm of Dumas, Pernet and Sedoglavic [2, 3] in its 262-operation form [6, 16] (`D48`), and their accurate $\langle 3 \times 3 \times 6 : 40 \rangle$ algorithm [4, 6] in its three

orientations $3 \times 3 \times 6$, $3 \times 6 \times 3$ and $6 \times 3 \times 3$ (`R336`, `R363`, `R633`). The three orientations applied in turn divide a square problem by 54 and leave it square, where three 4×4 levels divide by 64. A plan such as `SW/R363+3/R633/G6` is one Strassen–Winograd level, then a $3 \times 6 \times 3$ level whose products are pipelined on three pass threads, then a $6 \times 3 \times 3$ level whose 40 products run as six single-threaded GEMM streams. Mixing pays for three reasons that the measurements separate:

1. *Order.* A level writes more data than it reads (the 4×4 scheme three times as much, Strassen's 1.75 times); the larger expansion belongs at the smaller level, so `SW/D48/G` moves $9.75\,n^2$ words per side where `D48/SW/G` moves $12.25\,n^2$ with the same 336 products.

2. *Shape.* A rectangular level hands down rectangular products; the $3 \times 3 \times 6$ cycle keeps 0.741 of the products per level against the 4×4 scheme's 0.75, moves $11.89\,n^2$ words against 12, stays under 58 streams per pass against 64, and after two levels its products are larger, so Accelerate runs them 13% faster.

3. *Accuracy.* Each level multiplies the leaf error by a factor set by the scheme and the point chosen in its orbit; the $3 \times 3 \times 6$ algorithm's factor is 0.74–0.79 of the 4×4 algorithm's per level in f32, and both are below one, because their products are shorter dot products than the GEMM's. Under an error budget an accurate level admits deeper, faster plans.

Two facts about the costs organize everything. First, a fused pass is bound by memory traffic, not arithmetic: the 201-operation program we found for the 4×4 scheme [15], DPS's original 315-operation program and their 284-operation accurate program all run at the same speed as fused passes (totals within 1%), so the operation counts that scheme searches minimize have little value here, while the error growth of the program matters [16]. Arithmetic shows only when a program is wasteful: the direct sums of the $3 \times 3 \times 6$ scheme's R and P matrices take 18 operations per output and their passes ran 30% slower than the traffic predicts, which PLinOpt's optimizer removed. Second, the leaf GEMM dominates the time (85–95% of a plan), so a level pays only when its products are large enough for the GEMM to run near its peak, and a scheme that saves products beats one that saves additions. The ternary $\langle 3 \times 3 \times 6 : 42 \rangle$ scheme of Section 7 is the controlled experiment: a third of the pass arithmetic, two products more, and it loses.

Contributions. (i) A cost model of fused levels in blocks moved and GEMMs priced inside the level that sits on them, calibrated per machine and precision, with predictions within 0.3–3% at $n \geq 8192$ (Section 2). (ii) A screen of 70 published schemes by saving per block moved and by leaf error amplification per 4×4-equivalent saving, which leaves the accurate $\langle 3 \times 3 \times 6 : 40 \rangle$ cycle as the one addition to the 4×4 menu and rules out every larger shape and every Kronecker composite (Section 3). (iii) A per-level error model, validated within 25%, and the orbit floors of the amplification for the two accurate classes (Section 4). (iv) A planner over (m, k, n) shapes with GEMM streams, pipelines, padding to multiples of $2^a 3^b$, an error budget, and shape estimates that price uncalibrated rectangular problems within 2% (Section 5). (v) Measurements on the M4 Pro: 1.73× and 1.91× over Accelerate at $n = 13824$ in f32 and f64, with less f32 error than the GEMM; rectangular problems at 1.04–1.52× (Section 6). (vi) The ternary trial (Section 7). (vii) Two implementations, C and Rust, that agree in plans, predicted times and results bit for bit (Section 8). Code, programs, calibrations and raw results are in the repository [17].

Setting. Row-major $A \in \mathbb{R}^{m \times k}$, $B \in \mathbb{R}^{k \times n}$, $C = AB$, in f32 or f64. The machine is an Apple M4 Pro (two performance clusters of five cores, one efficiency cluster of four, 64 GB) running macOS 26; Accelerate's GEMM runs on the SME matrix units, one per cluster, at about 3.3 TFLOP/s

in f32 and 0.85 in f64 on large square problems. Times are the best of five interleaved rounds (each round runs every method once; medians are within 2% unless stated). Errors are max-norm, relative to $\max|C|$, on uniform random inputs: "relerr" against Accelerate's GEMM in the same precision, "err64" against the f64 product of the same inputs (for f32), so that the GEMM's own error appears too.

2 Fused levels and what they cost

2.1 Schemes as linear programs

A scheme $\langle g_1 \times g_2 \times g_3 : r\rangle$ is three matrices $L \in \mathbb{R}^{r\times g_1g_2}$, $R \in \mathbb{R}^{r\times g_2g_3}$, $P \in \mathbb{R}^{g_1g_3\times r}$ with

$$\operatorname{vec}(AB) \;=\; P\big((L\,\operatorname{vec}A)\circ(R\,\operatorname{vec}B)\big) \qquad \text{for all } A\in\mathbb{R}^{g_1\times g_2},\; B\in\mathbb{R}^{g_2\times g_3}, \tag{1}$$

$\circ$ the entrywise product and vec row-major. Applied to blocks, the same identity holds with A's g_1g_2 blocks of size $\frac{m}{g_1}\times\frac{k}{g_2}$, B's g_2g_3 blocks of $\frac{k}{g_2}\times\frac{n}{g_3}$, and C's g_1g_3 blocks of $\frac{m}{g_1}\times\frac{n}{g_3}$: the r products are $(\frac{m}{g_1}\times\frac{k}{g_2})(\frac{k}{g_2}\times\frac{n}{g_3})$ matrix products, computed by the plan below. A *level* is the application of (1) once; the matrices are realized as straight-line programs (PLinOpt's `.slp` syntax [6]), one per side, and the three programs are what a level executes:

- the L pass reads the g_1g_2 blocks of A and writes the r left factors S_i;
- the R pass reads the g_2g_3 blocks of B and writes the r right factors T_i;
- the products $M_i = S_iT_i$ run (GEMMs, or a level below);
- the P pass reads the r products and writes the g_1g_3 blocks of C.

Each pass is *fused*: one loop over the element positions of a block, with the whole program evaluated on scalars in registers at each position. A pass therefore reads every input block once and writes every output block once, and the program's operation count costs only arithmetic, which the memory system hides.

2.2 The cost of a level

Write w for the word size and B for the streaming bandwidth the passes reach. A level on an $(m\times k)(k\times n)$ problem moves

$$W(m,k,n) \;=\; (g_1g_2+r)\,\frac{mk}{g_1g_2} \;+\; (g_2g_3+r)\,\frac{kn}{g_2g_3} \;+\; (g_1g_3+r)\,\frac{mn}{g_1g_3} \tag{2}$$

words through its three passes, that is $n^2\big(3+r(\frac{1}{g_1g_2}+\frac{1}{g_2g_3}+\frac{1}{g_1g_3})\big)$ on a square problem, and costs

$$T(m,k,n) \;=\; r\,T_{\text{below}}\Big(\frac{m}{g_1},\frac{k}{g_2},\frac{n}{g_3}\Big) \;+\; \frac{w\,W(m,k,n)}{B}. \tag{3}$$

Two numbers of a scheme decide the second term: its *traffic* W/n^2 and its *streams*, the number of blocks a pass reads and writes at once, g_1g_2+r, g_2g_3+r and g_1g_3+r. Beyond about 64 streams a pass loses bandwidth: a $\langle 5\times5\times5:93\rangle$ level's 118-stream passes ran at half the bandwidth of the 64-stream passes of the 4×4 scheme. Table 1 gives these numbers for the menu. The first term is decided by the *saving* $r/(g_1g_2g_3)$ and by the GEMM's rate on the products' shape and size: the f32 GEMM keeps 3.2–3.3 TFLOP/s down to 2048 and drops to 3.0 at 1024 and 2.6 at 512, where the f64 GEMM keeps its 0.81–0.85 TFLOP/s down to 1024, so f64 rewards depth and f32 does not.

level	scheme	saving	streams $L/R/P$	traffic (n^2)	amp_{eq}	γ_{eq}
`SW`	Strassen–Winograd $\langle 2\times 2\times 2 : 7\rangle$	0.875	11/11/11	8.25	10.66	19.8
`D48`	DPS accurate $\langle 4\times 4\times 4 : 48\rangle$, 262-op. point	0.750	64/64/64	12.00	2.828	5.657
`R336`	DPS accurate $\langle 3\times 3\times 6 : 40\rangle$	0.741	49/58/58	11.89	2.280	6.162
`R363`	the same, orientation $3\times 6\times 3$	0.741	58/58/49	11.89	2.453	5.125
`R633`	the same, orientation $6\times 3\times 3$	0.741	58/49/58	11.89	2.280	6.162
`T336`	ternary $\langle 3\times 3\times 6 : 42\rangle$ (Section 7)	0.778	51/60/60	12.33	6.407	11.07
`T363`	orientation $3\times 6\times 3$	0.778	60/60/51	12.33	6.166	9.737
`T633`	orientation $6\times 3\times 3$	0.778	60/51/60	12.33	8.061	22.84

Table 1: The levels: saving of products per level, blocks moved by each pass, pass traffic on a square problem in units of n^2 words, and the two accuracy measures of Section 3, normalized to the classical algorithm of the shape and raised to the power $\ln 0.75/\ln(\text{saving})$ so that they are comparable per 4×4-equivalent saving.

2.3 What the implementation taught

The passes reach 120–135 GB/s against a peak of about 270. Two details were each worth a large factor. With 64 streams the compiler gave up vectorizing the fused loop (it could not rule out overlap among 64 pointers); a `vectorize(assume_safety)` pragma gave 4.3× on the passes. And scratch blocks of power-of-two size all start at the same offset modulo the cache's set span, so element c of all 64 streams fell into one cache set; padding each block by 1040 elements gave another 2.5×. Operation counts did not matter: our 201-operation program for the 4×4 scheme [15], DPS's 315-operation original and their 284-operation accurate program run at the same speed as fused passes (totals within 1%; the 315-operation passes take 2–8% longer, about 0.3% of the total). The exception is a wasteful program: the direct sums of the $3\times 3\times 6$ scheme's R and P matrices cost 18 operations per output, and their passes ran at 0.092 s against 0.070 for the 4×4 level's at 13824 in f32 for 0.99× the words. PLinOpt's optimizer [6] brought the nine programs of the three orientations from 152, 344+384 and 366+384 operations (L, R, P of the $3\times 3\times 6$ orientation; additions plus scalings) to 59, 117+22 and 148+18, which cut the R-level passes by a quarter and the plans by 1–7% with the error unchanged.

2.4 Using the whole machine

Accelerate drives each performance cluster's SME unit from one thread, so its GEMM keeps two or three cores busy and leaves the rest idle. The r products of a level are independent, and a leaf `G`k runs them as k single-threaded GEMM streams taking products in turn: three streams beat Accelerate's own threading by 10% (f32) and 8% (f64) at 2048, more streams win for the smallest products (f32 at 1024, f64 at 512) and lose badly for large ones (six streams +26–34% in f64 at 1024). The calibration times each level's products with 0, 3, 4 and 6 streams and the planner chooses per plan: 6–14% off the best plans.

A streaming pass beside the SME streams slows them, but less than the pass costs run alone: at f64 blocks of 512 two pass threads slow six streams by 14% while the pass runs at 49 GB/s, about 35% of its stand-alone cost. So a level whose products are themselves levels over GEMM streams can *pipeline* them (written `X+p`, p pass threads): while the streams compute sub-problem q, the pass threads write sub-problem $q-1$'s result and the factors of $q+1$, the scratch below double-buffered. Pipelining takes 8–22% off the plans over six-stream leaves; one pass thread is too few, and sub-problems under 1 ms are not pipelined. The calibration times a pipeline of 2 and of 10 synthetic sub-problems per level, block size, stream count (3, 6) and pass threads (2, 3, 4): the

scheme	saving	streams	traffic	amp_{eq}	γ_{eq}
$\langle 3\times3\times6:40\rangle$ DPS accurate (and $6\times3\times3$)	**0.741**	49/58/58	**11.89**	**2.280**	6.162
$\langle 3\times6\times3:40\rangle$ DPS accurate	0.741	58/58/49	11.89	2.453	**5.125**
$\langle 4\times4\times4:48\rangle$ accurate, 262-op. point	0.750	64/64/64	12.00	2.828	5.657
$\langle 3\times4\times6:54\rangle$ AlphaEvolve (±1 point)	0.750	66/78/72	12.75	4.637	6.784
$\langle 3\times4\times7:63\rangle$ rational	0.750	75/91/84	13.50	5.734	11.13
$\langle 5\times5\times5:93\rangle$ AlphaEvolve	0.744	118/118/118	14.16	8.560	17.46
$\langle 5\times5\times6:110\rangle$ Kemper	0.733	135/140/140	14.73	10.38	22.9
$\langle 6\times6\times6:153\rangle$ Moosbauer–Poole	0.708	189/189/189	15.75	9.229	18.3
$\langle 2\times4\times5:32\rangle$ AlphaEvolve	0.800	40/52/42	11.80	4.962	10.95
$\langle 2\times2\times2:7\rangle$ DPS accurate (in $\mathbb{Q}(\sqrt{3})$)	0.875	11/11/11	8.25	3.855	10.53
$\langle 2\times2\times2:7\rangle$ Strassen / Winograd	0.875	11/11/11	8.25	6.89 / 10.66	14.1 / 19.8
$\langle 3\times3\times3:23\rangle$ best of four	0.852	32/32/32	10.67	12.7	34.1
$\langle 3\times3\times6:40\rangle$ Smirnov (±1 point, same class)	0.741	49/58/58	11.89	9.754	26.1

Table 2: The menu screen (`matmul/levels/levels.py`): the schemes that could matter, at their published points. The accuracy columns are per 4×4-equivalent saving.

slope is the steady time per sub-problem, the rest the fill and drain. NEON GEMMs on the idle cores were tried and not adopted: at most 1–3%, fragile, and the SME units lose most of what the vector units add.

3 The menu: which schemes can help

In a fused-pass library a plan already is a combination of sub-schemes, one per level on the current (m,k,n) sub-problem, so the question of mixing is the question of the menu. We scored every scheme on disk – AlphaEvolve's 15 shapes [11], the 29 rectangular schemes of Kauers and Wood [8] and the $5\times5\times6$ and $6\times6\times6$ schemes of Moosbauer, Poole and Kemper [9], both as distributed with the addition-reduction library of Mårtensson and Stankovski Wagner [10], PLinOpt's small and rectangular schemes including DPS's accurate variants [6], Smirnov's $\langle 3\times3\times6:40\rangle$ [14], and Kronecker composites of the accurate pieces – by the quantities that decide a level's cost and error in (3):

- saving $r/(g_1g_2g_3)$, streams g_1g_2+r, g_2g_3+r, g_1g_3+r, and traffic $3+r\sum 1/(g_ig_j)$;
- the amplification of the leaf error,

$$\text{amp} \;=\; \max_j \; \tfrac{1}{2}\sqrt{\textstyle\sum_i P_{ji}^2\,\|L_i\|_2^2\,\|R_i\|_2^2}, \tag{4}$$

  the root-sum-square growth of independent rounding errors in the r products through output j, normalized by the classical algorithm's $\frac{1}{2}\sqrt{g_2}$; and DPS's growth factor $\gamma_{\infty,2} = \max_j \sum_i |P_{ji}|\,\|L_i\|_2\|R_i\|_2$ [1, 3], normalized by g_2.

Schemes of different shapes save different fractions per level, so the normalized factors are raised to the power $\ln 0.75/\ln(\text{saving})$, the growth per 4×4-equivalent saving, and are then directly comparable with the 4×4 scheme's 2.828 and 5.657 (the columns amp_{eq} and γ_{eq} of Table 1). The complete table is `matmul/levels/levels.csv` [17]; Table 2 shows the candidates.

Three conclusions. Everything larger than 4×4 saves more per level but needs 110–225 streams per pass, where the passes run at half bandwidth, and has 8–100× the classical amplification at its published points. Kronecker products add nothing: a composite keeps the per-saving amplification

of its pieces ($\langle 3\times3\times6\rangle \otimes \langle 3\times3\times6\rangle$ keeps 2.280, the accurate 2×2 squared keeps 3.855) and multiplies their stream counts, so it is never better than its two levels run one after the other and the stream explosion makes it worse. And the one scheme that beats the 4×4 scheme on every count but γ_{eq} is DPS's accurate $\langle 3\times3\times6 : 40\rangle$: applied to a square problem, the $3\times3\times6$ orientation hands down $(n/3, n/3, n/6)$ products, the $3\times6\times3$ orientation on those gives $(n/9, n/18, n/18)$, and the $6\times3\times3$ orientation returns to the square $(n/54)^3$. The cycle keeps 0.741 of the products per level, moves $11.89\,n^2$ words, stays within 58 streams, and amplifies the leaf error by 2.28–2.45 per 4×4-equivalent saving against 2.83. Perminov's catalogue of 680 shapes up to $16\times16\times16$ [12] confirms the menu: only $\langle 3\times3\times6 : 40\rangle$, $\langle 4\times4\times4 : 48\rangle$ and $\langle 3\times4\times6 : 54\rangle$ save a quarter of the products within 80 streams per pass, and the new shapes that save a quarter, $\langle 3\times6\times8 : 108\rangle$ and $\langle 4\times7\times9 : 187\rangle$, need 126–215 streams.

The paper cost model (3) with the GEMM at the lone 8192^3 f32 rate and the passes at the 4×4 level's bandwidth puts the cycle 1–3 points ahead of the 4×4 plans at every depth (`R336/G` 17.8% off the GEMM against `D48/G` 16.8%; `R336/R363/G` 13.9% against 10.8%), and the assumption behind it holds: Accelerate on the rectangular leaves at equal flops runs within 5% of the square GEMM after one level and 13% ahead after two, because the leaves are larger ($n/54$ against $n/64$ after three levels).

4 Accuracy

4.1 What a level does to the error

The rounding errors of the r leaf products pass through P into the outputs; the roundings of the passes themselves add to them. Taking the leaf errors as independent and proportional to the sizes of the factors gives (4): a level multiplies the leaf error by amp, against $\frac{1}{2}\sqrt{g_2}$ for the classical algorithm on the same blocks. This root-sum-square measure, not the worst-case sum $\gamma_{\infty,2}$, is what the measurements follow: the 4×4 orbit's $\gamma_{\infty,2}$ minimum (21.41, in $\mathbb{Q}(\sqrt{2})$) has amp $= 3.14$ and measures no better than DPS's Table 1 point, while the 262-operation point, with amp $= 2\sqrt{2}$ the orbit's minimum and the smallest $\gamma_{\infty,2}$ among dyadic points, measures 0.75–0.77 of it [16].

Over its whole orbit under $GL_3\times GL_3\times GL_6$ the $\langle 3\times3\times6 : 40\rangle$ class has amplification ratio at least 2.242 in every orientation (L-BFGS from random starts, `orbit_rect.py`); DPS's $3\times3\times6$ point is 5% above that floor and 5% above its $\gamma_{\infty,2}$ floor, their $3\times6\times3$ point 14% above (and 41% above that orientation's $\gamma_{\infty,2}$ floor of 3.90, far below the 4×4 class's 5.66). The ±1 Smirnov point of the same class (amplification ratio 10.8) reaches the same floors, so the class carries them, not the point. Per 4×4-equivalent saving the floor is 2.17 against the 4×4 class's 2.83, which is its orbit's minimum and, as far as continuous search from many starts can tell, a local minimum of the whole variety of rank-48 decompositions. A better exact point for the $3\times6\times3$ orientation is worth a dyadic search.

4.2 The error model and its validation

A plan's error is well described by the classical error scale times one factor per fused level, growing like $\sqrt{k}$ with the inner dimension,

$$\text{err}(\text{plan};\, m,k,n) \;=\; e_0\,\sqrt{k/N_E}\;\prod_{\text{levels } X} f_X, \tag{5}$$

with the order of the levels immaterial (measured: `SW` then `D48` and `D48` then `SW` within 2%) and streams and pipelines leaving the arithmetic unchanged. The calibration measures six plans at

$N_E = 2304$ (divisible by 16, 3 and 6) against the classical GEMM and fits e_0 and the factors: in f32 `SW` 1.09, `D48` 0.91, `R336` 0.74, `R363` 0.79, `R633` 0.76; in f64 1.99, 1.20, 1.25, 1.27, 1.42. The predictions hold within 25% from 4096 to 16384 except for two Strassen levels, over-predicted by up to 50% (the safe side). The accurate levels' factors are below one in f32: against the GEMM their products are k/g_2-long dot products instead of k-long ones, and the GEMM's own accumulation error is the larger term. Measured against the f64 product, one or two accurate levels have less error than Accelerate's f32 GEMM (Table 3). The budget `auto:`g takes the fastest plan whose predicted growth $\prod f_X$ is at most g; with the accurate levels every plan the planner likes qualifies down to $g = 2$, where the first 4×4 program we used, our 201-operation point with factor 3.4 per level, forced the classical blocked GEMM at $g = 2$ and one level at $g = 8$.

4.3 Programs and precision

For a fixed scheme the program still matters, through the roundings of the P pass: a first-order rounding model with exact second moments of every intermediate shows that DPS's 262-operation programs collect 15 roundings of RMS size 0.80 of the output per output where our 445-operation programs for the same point collect 36 of size 1.39, and among four programs of equal amplification the model's worst-output error orders the measured errors exactly at three and five levels [16]. The lever is nearly exhausted for the 4×4 class: an 11% smaller modelled pass error was invisible at one to three levels and worth about 5% at depth. The precision lever remains: running the passes of the f32 path in f64 (the blocks stay f32, the arithmetic is f64, one rounding at the GEMM interface) costs $\leq 2\%$ below 16384 and cuts the error by 36% at five levels, nothing at one to three, where the leaf GEMMs' error dominates. It is a switch in the library (`FMM_F64PASS`), not the default.

5 The planner

A plan is a sequence of levels from the top, ending in a leaf: `G` (one GEMM, Accelerate's threading), `G`k (k GEMM streams), or the classical blocked GEMM `B2` on its own. The planner is a dynamic program over the first level:

$$\mathrm{cost}(s) \;=\; \min\Big(\mathrm{leaf}(s),\; \min_X \big[\mathrm{passes}_X(s) + r_X\,\mathrm{cost}_X(s/g_X)\big]\Big), \tag{6}$$

where $s = (m, k, n)$, s/g_X is the shape of X's products, and $\mathrm{cost}_X(t)$ is the cheaper of a GEMM leaf *priced inside level* X (with 0, 3, 4 or 6 streams) and a further level on t. Pricing the leaf inside the level matters: a GEMM shorter than about 1 ms takes 13–27% longer called back to back, as inside a plan, than called alone, and the first calibration, which timed single calls, picked a two-level plan at 8192 f64 that ran 12% slower than the right one. A level whose sub-problems are a level over GEMM streams may instead be pipelined (Section 2.4), priced from the calibrated steady time per sub-problem plus fill and drain.

Calibration. Everything the model needs is measured once per machine and precision and cached (format 5, `calib5_f32.txt`, `calib5_f64.txt`, keyed by the machine, the padding, the file format and the programs the kernels came from): the lone GEMM and the blocked GEMM at each size; for each level and product shape its three passes and the GEMM time per product at 0, 3, 4 and 6 streams, measured together by running the level; the pipelines; and the six error-model plans at $N_E = 2304$. For n up to 13824 this is 408 entries and 13 minutes per precision. Square shapes interpolate (log-log between the measured sizes); rectangular shapes do not, and are either calibrated for the shape (118 entries, a few seconds) or priced by *shape estimates* from the square

calibration: GEMM-like entries at the square size of equal flops, $\sqrt[3]{mkn}$ (Accelerate runs rectangular shapes within 5% of the square rate), and a level's passes at the square size of equal traffic,

$$n_{\text{eq}} = \sqrt{\frac{(g_1 g_2 + r)\, mk + (g_2 g_3 + r)\, kn + (g_1 g_3 + r)\, mn}{g_1 g_2 + g_2 g_3 + g_1 g_3 + 3r}}, \tag{7}$$

the size at which a square problem moves the same words through the level's passes (2). The estimates predicted every measured rectangular case within 2% (Table 4).

Sizes and shapes the levels do not divide. A plan runs at the smallest shape its levels divide: every dimension is rounded up to the same multiple of $2^a 3^b$ ($a \le 10$, $b \le 3$; 44 candidates), the operands copied into zeroed matrices and the result copied back, priced at $2\times$ the words at 80 GB/s. The planner takes the cheapest padded candidate, so an n that 3 does not divide gets a 4×4 plan unless padding to a multiple of 54 pays. Planning a new shape costs 0.07 s in C and 0.010 s in Rust and is cached for 64 shapes.

Validation. Predictions are within 0.3–3% of the measured times at $n \ge 8192$ in both precisions throughout the development (first with the 4×4 menu, then with streams, pipelines and the $3\times3\times6$ levels), and the planner's choice was the fastest plan measured, or within 1% of it, at every size and precision tried, with two documented exceptions traced to noisy calibration entries (a pipeline slope from two and four sub-problems; a 4096 f32 run 15% slow throughout). The classical variants themselves drift between runs (13824 f64: 6.2–6.7 s), so speedups carry about $\pm5\%$.

6 Results

Table 3 compares, at two sizes and in both precisions, the single levels, the best plan the 4×4 menu offered, and the planner's mixed plan. What the table says:

- As a single level the $3 \times 3 \times 6$ scheme ties the 4×4 scheme at 13824 and is 9–19% more accurate; at 6912 in f32 it is 6% slower, because the passes are a larger share there and its L pass moves more (49 streams of larger blocks). The planner knows this and uses the orientation whose cheapest pass faces the largest operand.
- The mixed plans win in both time and error at every size from 1728 up. At 13824 f32, `R363+4/R363/G6` is 4% faster than the best 4×4 plan with 40% less error; the planner's top six plans there are all $3 \times 3 \times 6$-family plans. At 13824 f64, `SW/R363+3/R633/G6` runs $1.91\times$ over Accelerate against 1.83 for the best 4×4 plan.
- In f32 the mixed plans' error against the exact product is below the GEMM's own at every size (2.26 against $3.85 \cdot 10^{-6}$ at 6912, 2.46 against $6.40 \cdot 10^{-6}$ at 13824): a fast multiplication that is also the more accurate one.
- The three-orientation cycle alone (`R336/R363/R633/G`) does not pay at these sizes: its leaves of $n/54$ are too small for the GEMM. Depth pays through the pipelined two-level plans.

The planner's choices by size, from the calibration at 13824: f32 `G` at 1728, `R363/G6` at 3456, `R363+4/D48/G6` at 6912, `R363+4/R363/G6` at 13824; f64 `R363/G6` at 1728, `SW+3/R363/G6` at 3456, `R363+3/R633/G6` at 6912, `SW/R363+3/R633/G6` at 13824. Sizes that 3 does not divide keep their 4×4 plans (`D48+4/D48/G6` at 8192 f32, $1.40\times$; `D48+3/D48/G6` at 9000 f64 padded to 9008, $1.69\times$).

n, precision	plan	time (s)	over GEMM	relerr	err64
6912 f32	`G` (Accelerate)	0.195	1.00	–	$3.85 \cdot 10^{-6}$
	`D48/G`	0.154	1.27	$4.2 \cdot 10^{-6}$	$2.81 \cdot 10^{-6}$
	`R336/G`	0.164	1.19	$3.9 \cdot 10^{-6}$	$2.27 \cdot 10^{-6}$
	`D48+4/D48/G6`	0.149	1.31	$4.3 \cdot 10^{-6}$	$2.94 \cdot 10^{-6}$
	`R363+4/D48/G6` (auto)	**0.141**	**1.38**	$4.0 \cdot 10^{-6}$	$2.26 \cdot 10^{-6}$
13824 f32	`G` (Accelerate)	1.565	1.00	–	$6.40 \cdot 10^{-6}$
	`D48/G`	1.255	1.25	$7.1 \cdot 10^{-6}$	$4.16 \cdot 10^{-6}$
	`R336/G`	1.244	1.26	$6.7 \cdot 10^{-6}$	$3.80 \cdot 10^{-6}$
	`R336/R363/G`	1.200	1.30	$6.2 \cdot 10^{-6}$	$3.34 \cdot 10^{-6}$
	`D48+4/D48/G6`	0.949	1.65	$7.0 \cdot 10^{-6}$	$4.35 \cdot 10^{-6}$
	`R363+3/D48/G6`	0.906	1.73	$6.4 \cdot 10^{-6}$	$3.49 \cdot 10^{-6}$
	`R363+4/R363/G6` (auto)	**0.907**	**1.73**	$6.6 \cdot 10^{-6}$	$\mathbf{2.46 \cdot 10^{-6}}$
6912 f64	`G` (Accelerate)	0.773	1.00	–	
	`D48/G`	0.612	1.26	$7.8 \cdot 10^{-15}$	
	`R336/G`	0.597	1.29	$6.8 \cdot 10^{-15}$	
	`R336/R363/G`	0.594	1.30	$8.3 \cdot 10^{-15}$	
	`D48+4/D48/G6`	0.478	1.62	$7.9 \cdot 10^{-15}$	
	`R363+3/R633/G6` (auto)	**0.451**	**1.71**	$6.7 \cdot 10^{-15}$	
13824 f64	`G` (Accelerate)	6.223	1.00	–	
	`D48/G`	4.970	1.25	$1.40 \cdot 10^{-14}$	
	`R336/G`	4.946	1.26	$1.18 \cdot 10^{-14}$	
	`R336/R363/G`	3.703	1.68	$1.45 \cdot 10^{-14}$	
	`D48+4/D48/G6`	3.703	1.68	$1.38 \cdot 10^{-14}$	
	`SW/D48+2/D48/G6`	3.407	1.83	$1.85 \cdot 10^{-14}$	
	`SW/R363+3/R633/G6` (auto)	**3.251**	**1.91**	$1.51 \cdot 10^{-14}$	

Table 3: Square problems on the M4 Pro (`results/r336_campaign2_2026-10-28.txt`): best of five interleaved rounds; "over GEMM" is Accelerate's time divided by the plan's; relerr is the max-norm difference from Accelerate's product, err64 the f32 error against the f64 product (the GEMM's own error in the first row of each block). "auto" is the planner's choice.

Rectangular problems (Table 4) go through the same planner over (m, k, n): a level applies when its grid divides the current shape, so a $6912 \times 6912 \times 3456$ product gets a 4×4 level whose products are $1728 \times 1728 \times 864$ and then a $3 \times 6 \times 3$ level in f64. The shape estimates are within 2% of the measurements wherever a level pays, and where none does the planner correctly returns the GEMM (the first version of the estimates had none, and padded $1000 \times 3000 \times 2000$ to 3072^3, 30 times slower than Accelerate).

7 The ternary trial: cheap passes against accurate ones

Perminov's catalogue [12, 13] lists, beside the ranks, the schemes with coefficients in $\{-1, 0, 1\}$ whose additive complexity his parallel common-subexpression search reduced; for $3 \times 3 \times 6$ it has four rank-42 ternary schemes reduced to 139, 140, 145 and 150 additions for the three linear forms together (file `3x3x6_m42_cr139_cn287_ZT_reduced.json` and its siblings; the naive counts are 287–369). Against DPS's accurate rank-40 scheme, whose nine optimized programs take 374, 383 and 353 operations per orientation, this is the opposite design: two products more (saving 0.778 against 0.741), two more streams per pass (51/60/60), and a third of the pass arithmetic. Is the

$m \times k \times n$	calibration	prec.	plan	predicted / measured (s)	over GEMM
$3456 \times 3456 \times 1728$	for the shape	f32	`R363/G6`	0.0117 / 0.0115	1.05
		f64	`R363/G6`	0.0366 / 0.0367	1.30
$1000 \times 3000 \times 2000$	estimated	f32	`G`	–	1.00
		f64	`SW/G4`	0.0133 / 0.0144	1.04
$2048 \times 4096 \times 1024$	estimated	f32	`G`	–	1.00
		f64	`D48/G6`	0.0177 / 0.0180	1.10
$6912 \times 6912 \times 3456$	estimated	f32	`SW+4/D48/G6`	0.0790 / 0.0796	1.26
		f64	`D48+4/R363/G6`	0.2578 / 0.2565	1.52

Table 4: Rectangular problems through the library's entry points (`results/rect_api_2026-10-28.txt`): the planner's plan, its predicted and measured time, and the speedup over Accelerate's GEMM on the same shape. "Estimated" shapes are priced from the square calibration by (7).

arithmetic worth anything once the passes are fused, and what does the accuracy cost?

The scheme. The reduced JSON is a straight-line program already (fresh variables for the common subexpressions, then the forms); each of the four schemes was expanded to its matrices and checked exactly against the Brent equations. The 139-addition scheme is the most accurate of the four by far (amplification ratio 5.07 in the $3 \times 3 \times 6$ orientation against 7.6–9.5 for the others), so it is the one tried; Table 1 gives its three orientations. Its programs are Perminov's forms for the three maps the $3 \times 3 \times 6$ orientation needs (L: 33 additions, R: 39, P: 67), and for the other orientations the same forms re-indexed where an orientation reuses a map and transposed by PLinOpt's `transpozer` where it needs the transposed map ($3 \times 6 \times 3$: 39, 43+6 negations, 66; $6 \times 3 \times 3$: 43+6, 33, 63), all nine checked by `SLPchecker` against the matrices. PLinOpt's optimizer (the fork's `-s 1 -Q -O 500`) on the same nine matrices finds 34, 41, 71, 41, 45, 67, 45, 34, 66 additions: one to four more than Perminov's reduction on every matrix, in 0.2 s each. The levels `T336`, `T363`, `T633` went into the C library beside the R levels, outside the planner's menu, and every plan containing them computes the product (3456, both precisions, including pipelined plans).

Measured. Table 5 answers both questions. The pass arithmetic is worth nothing: the T levels' passes take exactly the R levels' times (at 13824 f32, $L/R/P$ 0.036/0.020/0.017 s against 0.034/0.019/0.017; at 6912, 0.009/0.005/0.005 for both), because a fused pass is bound by the blocks it moves and the T level moves two more per pass. The two extra products cost what the rank says: the GEMM part of a single level is 5–7% longer, a single T level is 4–5% slower than its R counterpart at every size and precision (the one tie, `T363/G` at 6912 f32, is within the run's noise), a two-level T plan 8–10% slower, and the three-orientation cycle 13–16% slower. The accuracy cost is the amplification: 2.2–3.5× the R level's error as a single level, 3.7–4× in two-level plans and 6.5–7× in the cycle, in line with the ratio of the amplification factors ($5.07/2.36 = 2.1$ for the $3 \times 3 \times 6$ orientation, the measured 2.2–3.5 including the passes' own roundings). In f32 every T plan's error is above the GEMM's own (`T336/G` $1.28 \cdot 10^{-5}$ against $6.40 \cdot 10^{-6}$ at 13824) where every R plan's is below it. The planner's menu was left as it was: a level that is slower and less accurate than one already there on every plan cannot be chosen.

What it shows. The trial is the cleanest confirmation of the cost model's premise. Two schemes of the same shape, with the same pass structure, differ in rank by two and in pass arithmetic by a

n, prec.	plan	R (s)	T (s)	T/R	R error	T error	T/R
13824 f32	`X336/G`	1.228	1.286	1.05	3.80e-6	1.28e-5	3.4
	`X363/G`	1.242	1.304	1.05	2.73e-6	6.25e-6	2.3
	`X633/G`	1.276	1.339	1.05	3.95e-6	1.01e-5	2.6
	`X336/X363/X633/G`	2.538	2.874	1.13	4.95e-6	3.53e-5	7.1
	`X363+4/D48/G6`	0.909	0.954	1.05	3.49e-6	6.23e-6	1.8
	`X363+4/X363/G6`	0.906	0.994	1.10	2.46e-6	9.04e-6	3.7
13824 f64	`X336/G`	4.858	5.089	1.05	1.18e-14	2.62e-14	2.2
	`X363/G`	4.919	5.132	1.04	1.30e-14	1.34e-14	1.0
	`X336/X363/X633/G`	7.546	8.763	1.16	1.24e-14	8.08e-14	6.5
	`SW/X363+3/X633/G6`	3.236	3.573	1.10	1.51e-14	5.61e-14	3.7
	`X363+3/X633/G6`	5.624	6.188	1.10	1.26e-14	2.63e-14	2.1
6912 f32	`X336/G`	0.165	0.173	1.05	2.27e-6	8.00e-6	3.5
	`X363/G`	0.158	0.160	1.01	1.76e-6	3.87e-6	2.2
	`X363+4/D48/G6`	0.142	0.147	1.04	2.26e-6	4.07e-6	1.8
	`X363+3/X633/G6`	0.160	0.173	1.08	2.20e-6	8.72e-6	4.0
6912 f64	`X336/G`	0.598	0.626	1.05	6.78e-15	1.47e-14	2.2
	`X363+4/D48/G6`	0.453	0.478	1.06	7.03e-15	8.81e-15	1.3
	`X363+3/X633/G6`	0.451	0.497	1.10	6.74e-15	1.63e-14	2.4

Table 5: The ternary $\langle 3 \times 3 \times 6 : 42 \rangle$ levels (T) against the accurate $\langle 3 \times 3 \times 6 : 40 \rangle$ levels (R) on the same plans (`results/t42_campaign_2026-10-28.txt`; `X` stands for `R` or `T`): best of five (f32) or three (f64) interleaved rounds, seconds; the f32 error is against the f64 product, the f64 error against Accelerate's product.

factor of 2.7; the measured difference is the rank's, to the percent, and none of the arithmetic's. For a fused-pass library the objectives of a scheme search are rank and amplification, in that order, and additive complexity is not among them. The caveat is the one stated in Section 2: arithmetic shows when a program is wasteful enough to make a pass compute-bound, as the 18-operation-per-output direct sums were; Perminov's 139 additions and DPS's optimized programs are both well inside the bandwidth-bound regime. Whether the class of the ternary scheme contains an accurate point is a separate question, and the answer is no: over $GL_3 \times GL_3 \times GL_6$ the class's amplification ratio is at least 2.825 in the $3 \times 3 \times 6$ orientation and 2.831 in $3 \times 6 \times 3$ (eight starts each), the 4×4 class's $2\sqrt{2}$ and 26% above the accurate $\langle 3 \times 3 \times 6 : 40 \rangle$ class's 2.242; the 139-addition point sits 1.8× above its own floor, and its $\gamma_{\infty,2}$ floors (5.12 and 4.82) are likewise above the accurate class's (6.37 and 3.90) in the orientation that matters. Even at its best point the class would match the 4×4 scheme's accuracy with two products more than the accurate $3 \times 3 \times 6$ scheme.

8 Two implementations

The library is C (`fmm.h`, `libfmm.a`: `fmm_sgemm`, `fmm_dgemm` and their `_mkn` forms for an $m \times k$ times $k \times n$ product with separate leading dimensions, `fmm_calibrate`, `fmm_plan`, `fmm_release`) over Accelerate, with the kernels generated from the schemes' programs by `gen_kernels.py`: every program in PLinOpt's syntax becomes one fused loop in f32 and in f64, plus an f64-arithmetic variant for f32 blocks. Plans are made once per (shape, precision, budget) and kept; the workspace is kept between calls; a call outside the calibrated range or without a calibration runs the classical GEMM, so a call is always correct. The benchmark driver includes the library, so both run the same code, and every plan prints its predicted time next to the measured one.

The Rust port (crate `fmm`, `rust/`) is the same library module for module – plans and shapes,

the calibration in the C file format so that either library reads the other's cache, the planner with pipelines, budget and padding, the executor on a rayon pool, the entry points – with the fused passes as safe Rust emitted by the same generator (each slice a separate parameter, cut to the row length first: the compiler then vectorizes as `restrict` and the pragma let it in C). The only unsafe code is the three FFI declarations for Accelerate behind safe wrappers that check the slices. Cross-checked against the C library on the shared calibration, 26 cases (square sizes from 1000 to 9000, a leading dimension above n, an error budget, four rectangular shapes, both precisions) give the same plans, the same predicted times to the digit and results bit for bit. Speeds match at large sizes (f64 8192: 0.801 against 0.815 s; f32 $6912 \times 6912 \times 3456$: 0.0789 against 0.0796); the Rust passes on blocks under about 600 are 1.2–2× slower (dispatch and per-chunk allocation), which only shows when Rust calibrates for itself. Two traps found on the way: a zeroed `Vec` gets its pages on first write, so timed buffers must be written first; and a lone multi-threaded GEMM issued right after GEMM streams on the rayon pool runs 15–25% slower and stays so, which the leaves' timing avoids by running where the library runs a classical plan.

9 Discussion

The mixed approach rests on two measurements that contradict the usual accounting. A fused level's cost is the blocks it moves, so the right measure of a scheme is its saving per block moved and its stream count, not its rank or the length of its programs; and the accuracy of a level is set by the point in its orbit and, at depth, by the roundings of its P program, not by the operation count. Under these measures the menu is short: two accurate schemes and Strassen–Winograd for the top of deep f64 plans. A scheme enters the menu only if a plan containing it beats every plan without it on this machine, and the planner decides that per shape and precision from measurements, which is why the library can take a new level (or a new machine) without any change but a calibration.

The limits are also measured. The GEMM is 85–95% of every plan, so no menu can take more than the products it saves, and below $n \approx 3000$ nothing pays in f32 (1728 in f64). A level's passes must not fall behind the streams it feeds: six streams of 1024-blocks in f64 are the most sensitive to pass traffic, and pipelining gains nothing there. Shapes above 4×4 cannot help because of the stream count, whatever their rank, and Kronecker composites because they inherit their pieces' amplification and multiply their streams. And the ternary trial (Section 7) closes the remaining door on the cheap side: when the passes are already bandwidth-bound, a program with a third of the additions buys nothing, while two extra products and 2.8× the amplification cost speed and accuracy at every size.

What is next. A dyadic point of the $3 \times 6 \times 3$ orientation closer to its class's amplification floor (DPS's is 14% above it) would improve the plans that use it; the tools exist from the 4×4 search. Fusing the L and R passes into GEMM operand packing, the "Strassen reloaded" design, needs an open GEMM kernel; on this machine the vendor's SME kernels are closed, so it waits for another machine (x86-64 with BLIS, or the Apple GPU). The planner's 0.07 s per new shape in C should be a millisecond. The library takes $C = AB$ only: α, β and transposes are a day's work. And the Rust passes on small blocks should be brought to the C library's speed.

10 Data and code

Everything is in the repository [17], directory `matmul/fastmm/`: the library (`fmm.c`, `fmm_impl.h`, `fmm.h`), the kernel generator (`gen_kernels.py`), the programs of the $3 \times 3 \times 6$ levels (`schemes/opt/r*`, PLinOpt-optimized; `schemes/r*`, the direct sums) and of the ternary levels (`schemes/`

`t*`), the Rust crate (`rust/`), the benchmark and the test (`bench_fmm.c`, `test_fmm.c`), the calibrations (`results/calib5_f32.txt`, `calib5_f64.txt`) and the raw results of every run quoted here: `results/r336_campaign_2026-10-28.txt`, `r336_campaign2_2026-10-28.txt` (Table 3), `r336_opt_vs_direct_2026-10-28.txt`, `rect_api_2026-10-28.txt` (Table 4), `t42_campaign_2026-10-28.txt` (Table 5), and the earlier campaigns of October 24–26 (streams, pipelines, accuracy, the 4×4 programs). The menu screen is `matmul/levels/` (`levels.py`, `levels.csv`, `orbit_rect.py`, `cost_rect.py`, README). The accuracy study of the 4×4 class is `doc/matmul_4x4_rank48_accurate262.pdf` [16]. The running log of the track, with every measurement in the order it was made, is `doc/fastmm_plan.md`.

References

[1] J.-G. Dumas, C. Pernet, A. Sedoglavic, *Strassen's algorithm is not optimally accurate*, Proc. ISSAC 2024, 254–263.

[2] J.-G. Dumas, C. Pernet, A. Sedoglavic, *A non-commutative algorithm for multiplying* 4×4 *matrices using* 48 *non-complex multiplications*, arXiv:2506.13242 (2025).

[3] J.-G. Dumas, C. Pernet, A. Sedoglavic, *A more accurate rational non-commutative algorithm for multiplying* 4×4 *matrices using* 48 *multiplications*, arXiv:2603.18699 (v1 19 March 2026, v3 29 July 2026).

[4] J.-G. Dumas, C. Pernet, A. Sedoglavic, *Towards automated generation of fast and accurate algorithms for recursive matrix multiplication*, J. Symbolic Comput. 134 (2026) 102524.

[5] J.-G. Dumas, C. Pernet, A. Sedoglavic, J. Stapleton, P. Tichavský, *Fast matrix multiplication via recursive* $\langle 4 \times 4 \times 4 : 48 \rangle$ *algorithms into practice*, arXiv:2609.12027 (v1 10 September 2026, v2 2 October 2026).

[6] J.-G. Dumas, B. Grenet, C. Pernet, A. Sedoglavic, *PLinOpt, a collection of C++ routines handling linear & bilinear programs*, https://github.com/jgdumas/plinopt; data files `4x4x4_48_accurate_262` (commit `b441528`, 13 October 2026), `3x3x6_40_DPS-accurate`, `3x6x3_40_DPS-accurate`, `6x3x3_40_DPS-accurate`; tools `optimizer`, `transpozer`, `SLPchecker`.

[7] J. Huang, T. M. Smith, G. M. Henry, R. A. van de Geijn, *Strassen's algorithm reloaded*, Proc. SC '16 (International Conference for High Performance Computing, Networking, Storage and Analysis), 2016.

[8] M. Kauers, J. Wood, *Consequences of the Moosbauer–Poole algorithms*, preprint (2025).

[9] J. Moosbauer, M. Poole, *Flip graphs with symmetry and new matrix multiplication schemes*, preprint (2025).

[10] E. Mårtensson, P. Stankovski Wagner, *The number of the beast: reducing additions in fast matrix multiplication algorithms for dimensions up to 666*, Proc. ACDA 2025; the library `fmm_add_reduction`, https://github.com/werekorren/fmm_add_reduction (commit `951cd93`, 8 May 2025), whose `algorithms/` directory distributes the Kauers–Wood, Moosbauer–Poole–Kemper and AlphaEvolve schemes used in the screen.

[11] A. Novikov et al., *AlphaEvolve: a coding agent for scientific and algorithmic discovery*, arXiv:2506.13131 (2025).

[12] A. I. Perminov, *FastMatrixMultiplication*: a catalogue of fast matrix multiplication schemes, https://github.com/dronperminov/FastMatrixMultiplication, commit `3ee3284` (28 October 2026); file `schemes/results/addition_reduced_ZT/3x3x6_m42_cr139_cn287_ZT_reduced.json`.

[13] A. I. Perminov, *Parallel heuristic exploration for additive complexity reduction in fast matrix multiplication*, arXiv:2512.13365 (2025).

[14] A. V. Smirnov, *The bilinear complexity and practical algorithms for matrix multiplication*, Comput. Math. Math. Phys. 53 (2013) 1781–1795.

[15] G. Sidebottom, *201 operations for rank-48* 4×4 *matrix multiplication over rings containing* $1/2$, preprint (2026); PLinOpt data file `4x4x4_48_201`.

[16] G. Sidebottom, *The 262-operation* $\langle 4 \times 4 \times 4 : 48\rangle$ *algorithm is also the more accurate one: error-bound exponent* $9/4$ *and about a quarter less error*, draft of 27 October 2026, `doc/matmul_4x4_rank48_accurate262.pdf` in [17].

[17] G. Sidebottom, *logic*, source repository, https://github.com/gsidebottom/logic (2026).